PROJECT TECHNICAL SPECIFICATION & VIVA DOCUMENTATION

AirSense India: Architecture, Languages & Implementation Details

National Air Quality Index (AQI) Telemetry, CPCB NAAQS Calculation & GIS Platform

PROJECT AUTHOR	ACADEMIC SCOPE	ARCHITECTURE	STANDARD
Preethish	College Python Project	FastAPI + Modern Web ES6	CPCB • IQAir • US EPA

1. Project Overview & Problem Statement

AirSense India is a full-stack environmental monitoring platform developed to provide continuous, real-time air quality metrics, major pollutant telemetry, and regulatory air quality index (AQI) calculations across **all 28 Indian States and 8 Union Territories** covering 43 CAAQMS (Continuous Ambient Air Quality Monitoring Stations).

The platform solves a key public health challenge in India: translating raw, complex atmospheric pollutant sensor concentrations (µg/m³ and ppb for PM2.5, PM10, CO, SO2, NO2, O3, NH3) into an intuitive, actionable single integer (AQI scale 0–500), paired with clinical health guidelines, dynamic 3-state protective mascot advisories, a daily air quality calendar, and geospatial GIS maps.

2. Programming Languages Used & Technical Justification

The project leverages a carefully designed modern technology stack combining **Python** on the backend, **JavaScript (ES6+)** on the frontend, and **HTML5/CSS3** for semantic layout and design. Each language was deliberately selected for specific computational and architectural strengths:

1. Python (Version 3.10+)

Backend Server, Numerical Modeling & REST API Engine

Core Frameworks & Modules: FastAPI, Uvicorn, Pydantic, Requests, Math, DateTime, CSV.

Why Python was chosen for this software:

- **Mathematical & Scientific Precision:** Calculating CPCB sub-indices requires piecewise linear interpolation across 6 pollutant concentration breakpoints. Python's floating-point precision, mathematical standard library, and robust data structures allow clean, bug-free implementation of national environmental standards.
- **High-Performance Asynchronous API (FastAPI):** Built on ASGI (Asynchronous Server Gateway Interface) using Starlette and Uvicorn, FastAPI provides asynchronous concurrency capable of handling hundreds of concurrent telemetry requests with sub-10ms response latency.
- **Strict Type Safety & Validation with Pydantic:** The CPCB Calculator API endpoint (`/api/calculate`) receives user-submitted pollutant parameters. Pydantic models automatically validate data types, range bounds, and handle missing inputs gracefully.
- **Extensibility for Machine Learning:** Python is the undisputed industry standard for environmental data science and atmospheric modeling. It allows seamless integration with predictive models (such as 7-day ARIMA / Random Forest forecasts in `services/forecast.py`).
- **Telemetry Multi-Source Ingestion:** Python efficiently fetches and aggregates live outdoor data from external environmental APIs including IQAir AirVisual API, OpenWeatherMap Air Pollution API, and World Air Quality Index (WAQI).

2. JavaScript (Modern ECMAScript / ES6+) Client-Side Controller, Dual-Engine Architecture & UI Logic

Libraries & APIs: Leaflet.js (GIS Mapping), Google Maps Platform API, Bootstrap 5 JavaScript, DOM API, Canvas & Native Print Engine.

Why JavaScript was chosen for this software:

- **Autonomous Dual-Engine Execution:** The application was engineered to function both as a full-stack client-server app (with FastAPI) and as a 100% standalone static web app on GitHub Pages. JavaScript hosts a full local telemetry dataset (`INDIAN_CITIES_LIST`) enabling instant rendering without server latency or network outages.
- **Dynamic Mascot Health Logic:** JavaScript evaluates live AQI scores in real-time to drive the 3-state protective mascot character:
  - **AQI 1 to 50:** Displays unmasked boy mascot — air is clean and healthy.
  - **AQI 51 to 100:** Displays boy covering nose/mouth — sensitive groups take moderate precaution.
  - **AQI > 100:** Displays masked boy mascot — protective face mask (N95) is mandatory.
- **Interactive GIS Spatial Mapping:** Leaflet and Google Maps layers are dynamically rendered and managed in JavaScript, plotting 43 color-coded state stations with interactive popup telemetries and camera transitions.
- **Zero-Latency Client-Side CPCB Calculator:** Recalculates sub-indices, overall AQI, and dominant pollutant instantly as user inputs change without requiring constant round-trip server requests.
- **Native A4 PDF Document Generation:** JavaScript formats and opens an official, publication-ready Air Quality Telemetry Document and immediately launches the system print dialog targeting PDF.

3. HTML5 & CSS3 (Modern Web Standards)

Semantic Structure, Design System & Color Psychology

Frameworks & Toolkits: Bootstrap 5 CSS, FontAwesome 6, Google Fonts (Inter, Fira Code), CSS Grid & Flexbox.

Why HTML5 & CSS3 were chosen for this software:

- **Semantic Document Hierarchy:** HTML5 elements (`<nav>`, `<main>`, `<section>`, `<select>`, `<optgroup>`) ensure structured accessibility, intuitive tab navigation, and mobile viewport responsiveness.
- **Official CPCB Color Classification:** CSS classes map exactly to the 6 official CPCB risk color standards: **Good (0-50)**, **Moderate (51-100)**, **Poor (101-200)**, **Unhealthy (201-300)**, **Severe (301-400)**, and **Hazardous (401+)**.
- **Clean, Modern Aesthetics:** Replaced outdated dark backgrounds with an ice-silver and clean modern theme featuring glassmorphism frosted weather cards, rounded pill action controls, and responsive grid layouts.
- **Print Stylesheet (`@media print`):** Dedicated CSS ensures that PDF document exports print with clean margins (A4 portrait), sharp typography, and zero extraneous UI buttons.

3. Software Architecture & File Structure

AirSense India follows a modular **Service-Oriented Architecture (SOA)** where business logic, data models, calculations, and presentations are decoupled:

File / Module	Language	Architectural Responsibility & Functional Purpose
`app.py`	Python	FastAPI main server. Defines REST routes (`/api/city`, `/api/calculate`, `/api/calendar`), static file mounts, and CORS policies.
`services/aqi_calculator.py`	Python	CPCB National Air Quality Index engine. Implements standard 24-hr breakpoint tables, sub-index formulas, and dominant pollutant logic.
`services/pollution_index.py`	Python	Composite Pollution Index (CPI) algorithm computing weighted multi-pollutant health load across pollutants.
`services/data_service.py`	Python	Comprehensive database for all 43 Indian cities, GPS coordinates, CAAQMS baseline telemetries, and `resolve_city_key()` lookup resolver.
`services/forecast.py`	Python	Generates 7-day predictive meteorological and AQI trend estimates using seasonal regression multipliers.
`services/health_advisory.py`	Python	Generates medical precautions, clinical health advisories, and outdoor activity recommendations per AQI level.
`public/js/app.js`	JavaScript	Main client application logic: multi-tier city resolver (`findCity`), Leaflet map manager, calendar builder, and PDF report generator.
`public/index.html`	HTML5	Responsive single-page dashboard containing navigation, state selectors, hero section, mascot container, calendar, and map widgets.
`public/css/style.css`	CSS3	Design system, CPCB spectrum bar, frosted weather cards, responsive breakpoints, and print stylesheets.

4. Core Mathematical & Environmental Algorithms

CPCB Piecewise Linear Interpolation Formula for Sub-Index (I_p):
The sub-index for any pollutant concentration C_p is calculated using:

```
Ip = ((I_hi - I_lo) / (B_hi - B_lo)) * (Cp - B_lo) + I_lo Overall AQI = Max(I_pm25, I_pm10, I_no2, I_so2, I_co, I_o3, I_nh3)
```

Where:
• C_p = Observed pollutant concentration.
• B_{hi}, B_{lo} = Breakpoint concentrations greater than and less than C_p.
• I_{hi}, I_{lo} = Sub-index values corresponding to B_{hi} and B_{lo}.
• Minimum criteria: According to CPCB guidelines, at least 3 pollutants must be monitored with at least one being PM2.5 or PM10.

5. College Viva Voce & Project Defense Guide

Q1: Why did you choose FastAPI over Flask or Django?

Answer: FastAPI was chosen because it natively supports modern Python asynchronous coroutines (async/await) on ASGI, delivers high execution speed on par with NodeJS and Go, automatically generates OpenAPI documentation, and provides automatic request validation through Pydantic without bloated boilerplate.

Q2: How does the website work on GitHub Pages if GitHub Pages doesn't run Python?

Answer: The software is engineered with an intelligent *Dual-Engine Architecture*. When deployed on GitHub Pages, client-side JavaScript utilizes the embedded `INDIAN_CITIES_LIST` dataset and executes the CPCB formulas directly in the browser. When run locally with `uvicorn app:app`, it seamlessly upgrades to live API calls to the FastAPI backend.

Q3: How does the software resolve states and cities accurately?

Answer: The project features a custom multi-tier resolution engine (`findCity()` in JavaScript and `resolve_city_key()` in Python). It checks exact IDs, exact city and state names, compound queries (e.g. "Goa - Panaji"), and state alias keyword tables, ensuring queries never misidentify locations or default incorrectly.

Q4: How is the PDF document generated?

Answer: The **Print PDF** button invokes `window.printCityReport()`, which compiles the active city's verified telemetry into an official A4 document layout with customized medical mask advisories, full pollutant tables, and certification stamps, immediately launching the browser's print engine configured with `@page { size: A4 portrait; }` for zero-dependency PDF creation.